学案 03 | 从 LLM 到 Agent：系统视角

1. LLM 能做什么，不能单独保证什么

LLM（large language model，大语言模型）是一个根据输入计算语言输出的模型。它能生成文本，也可能在训练中学到有用的知识、格式和解题模式。但一个裸模型本身通常不会自动打开文件、执行命令、访问实时网页或保存跨会话记忆。

当模型回答问题时，答案来自当前输入、模型参数和推理设置。若信息需要实时数据或实际操作结果，系统必须提供相应的工具或环境，并把结果送回模型。模型流畅地说"我已经完成"不能当作执行证据。

2. 什么是 Agent

在本学习路线中，Agent 指围绕目标组织起来的系统：它能根据当前任务和观察结果选择下一步，在必要时使用工具，并依据工具反馈继续或结束。一个简化架构如下：

```
用户目标
    ↓
控制器 / 编排逻辑 ── 对话状态与任务状态
    ↓                               ↑
LLM 选择下一步                      │ 工具结果/环境观察
    ↓                               │
工具调用 → 程序校验与执行 → 外部工具/环境
    ↓
最终答复与外部验收
```

常见组成包括：

- **模型**：理解当前输入、生成内容或提出动作。
- **上下文与状态**：保存当前任务需要的历史、变量、步骤状态和工具结果。
- **控制器（controller/orchestrator）**：负责把消息发给模型、识别是否有调用、调度工具、限制轮数和判断何时结束。
- **工具（tools）**：被程序实际执行的函数或服务，例如只读文件查询、计算器、检索器或测试命令。
- **环境（environment）**：工具作用和反馈所处的外部世界，例如文件系统、网页、模拟器。
- **验证器/评估器**：检查输出或行动是否满足可观察标准，例如测试是否通过。

实际系统可能把多个职责合并，或只实现其中一部分。规划、记忆和工具使用是分析 Agent 的常见维度，不要求每个 Agent 都有三个彼此独立的模块。

3. 工具调用的职责边界

假设用户问："请计算 284×17，并说明过程。"模型可能产出一个计算器调用请求。系统中的实际链条是：

1. 模型提出工具名称和参数。
2. 控制器解析结构化请求，检查工具是否存在、参数是否合规。
3. 程序执行计算函数。
4. 程序把运算结果和状态回填到对话。

5. 模型依据回填结果组织答复，或继续提出动作。
6. 系统检查是否完成并停止。

模型提出调用不等于调用已经执行。程序执行成功也不等于模型正确解释了结果。因此轨迹里要分别记录“模型请求”“程序执行状态”“工具返回内容”和“最终验证”。

4. 三种容易混淆的“记忆”

1. **模型参数**：通过训练形成的长期数值，不会因普通对话自动修改。
2. **当前上下文**：本次请求中实际提供给模型的消息和材料，受上下文容量约束。
3. **外部持久化记忆**：应用自己保存并在需要时重新检索/加入上下文的数据，例如数据库中的用户偏好。

系统可以用外部存储实现跨会话记忆，但需要处理写入、检索、更新、冲突和隐私。它不是模型天然拥有的无限记忆。

5. Workflow 与动态 Agent

Workflow 通常按人预先安排的固定步骤执行，例如“先检索 → 再总结 → 最后检查格式”。**Agentic loop** 则允许模型根据反馈动态选择下一步，例如先查看文件，再决定修改和测试。很多实用系统把两者结合：固定外层流程，局部让模型选择工具。

动态程度越高，系统可能越灵活，同时也更难预测、评测和限制。因此不应为了“像 Agent”而强行使用复杂循环。任务固定时，简单工作流往往更容易测试。

6. 一个小任务的架构拆解

任务：“检查一个 CSV 的列名是否重复，并给出报告。”

- 模型负责：理解用户目标、请求读取或分析数据、解释结果。
- 控制器负责：把目标和可用工具送入模型，调度调用并控制停止。
- 工具负责：真正读取 CSV、统计重复列名。
- 环境负责：提供文件内容和执行结果。
- 验证负责：确认工具读的是指定文件，统计结果和原文件一致。
- 失败边界：文件不存在、编码错误、用户没有权限、工具超时、模型误述报告。

把职责拆清楚后，可以问：错误是任务理解错了、调用参数错了、工具执行错了，还是结果解释错了？这正是读 Agent 系统和轨迹时要培养的分析方式。

7. 自己检查

1. 为什么 LLM 单独不能证明它真的改过某个文件？
2. 模型提出工具请求后，谁负责实际执行？
3. 上下文、模型参数和外部持久记忆有什么区别？
4. 为什么固定 workflow 有时比动态 Agent 更合适？
5. 如果工具返回正确但最终答案错误，优先检查哪一段数据流？